

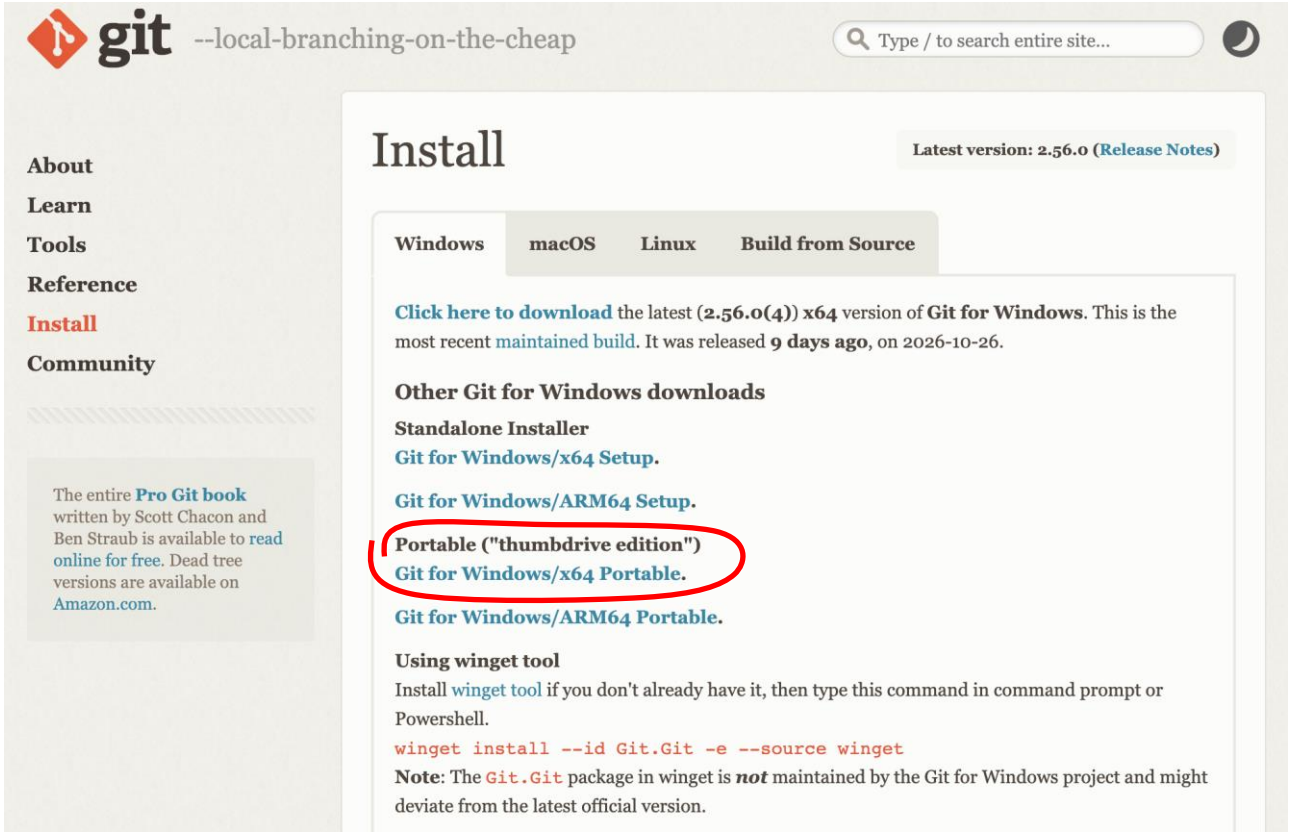
Engineering Programming

Workshop 10: Review

Dr. Hung Ta

Setup GIT

- Use portable version GIT
- https://git-scm.com/install/windows



git_cmd or git_bash

Setup the GIT

- Create digital signature
  - ssh-keygen -t ed25519 –C “h.ta@adelaide.edu.au”
  - Generate a key pair, public key and private key.
  - Go to:
    - github/your account/settings/SSH and GPG keys
    - Add public key to Authentication keys
  - Test connect with github
    - ssh -T git@github.com
  - Create a global config
    - git config - -global user.name “a3167030”
    - git config - -global user.email “h.ta@adelaide.edu.au”

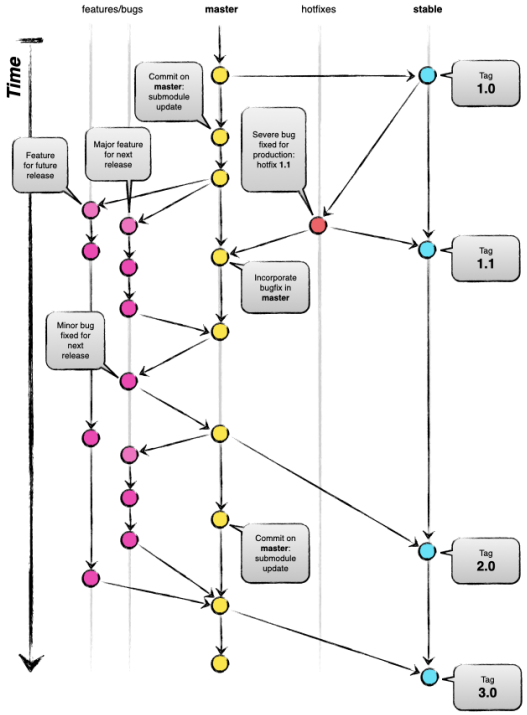
GITHUB

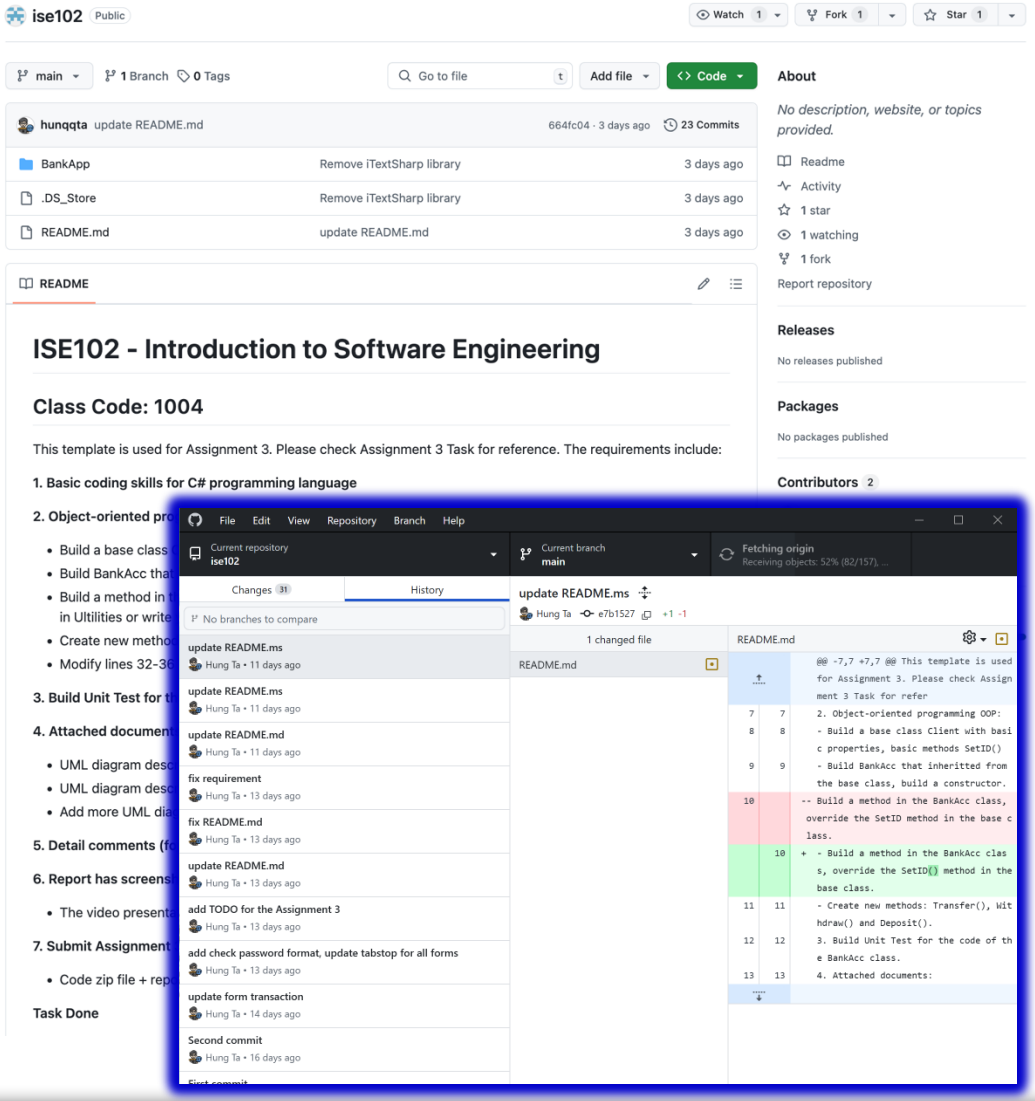
- https://github.com/ADLdev-SA/ise102/tree/main

Tools: GitHub desktop

CLI:

- git status
- git add .
- git commit -m "update code one"
- git push origin master/main
- git pull origin master/main





Testing - Pytest

- pytest –v

```
(base) b3@Atoms-MacBook-Pro Module10 % pytest -v
============================= test session starts ==============================
platform darwin -- Python 3.13.5, pytest-8.3.4, pluggy-1.5.0 -- /opt/anaconda3/bin/python
cachedir: .pytest_cache
rootdir: /Users/b3/Academic/AdelaideUni/Courses/COMP1006-Engineering Programming/Workshop/Module10
plugins: anyio-4.7.0
collected 1 item

test_bisection.py::test_bisection PASSED                                 [100%]

============================== 1 passed in 0.16s ===============================
(base) b3@Atoms-MacBook-Pro Module10 %
```

- pip install pytest-mock

pytest-mock

Simple and convenient mocking for pytest

pytest-mock is a plugin for `pytest` that provides a convenient fixture, *mocker*, which is a thin wrapper around Python's built-in `unittest.mock` module.

Less boilerplate. More focused tests.

- ☑ Mock easily
- ☑ Integrate with pytest
- ☑ Write cleaner tests
- ☑ Focus on behavior

What is it?

pytest-mock adds a *mocker* fixture to your pytest tests, making it easy to create and manage mocks (spies, stubs, patches, etc.).

Why use it?

- ✓ Simpler and cleaner syntax than using unittest.mock directly
- ✓ Integrates seamlessly with pytest
- ✓ Automatic cleanup of mocks between tests
- ✓ Supports common mocking patterns (patch, spy, stub, etc.)

Installation

```
pip install pytest-mock
```

Example

Suppose we have a simple function that calls an external API:

app.py

```python
import requests

def get_user_name(user_id):
    response = requests.get(f"https://api.example.com/users/{user_id}
    data = response.json()
    return data["name"]
```

In our test, we can use the `mocker` fixture to mock the API call:

test_app.py

```python
def test_get_user_name(mocker):
    # Mock the requests.get call
    mock_response = mocker.Mock()
    mock_response.json.return_value = {"name": "Alice"}
    mocker.patch("app.requests.get", return_value=mock_response)

    from app import get_user_name
    name = get_user_name(123)
    assert name == "Alice"
    # Verify the mock was called
    mocker.assert_called_once()
```

Common mocker methods

`mocker.Mock()`	Create a mock object
`mocker.patch()`	Replace an object with a mock (works like unittest.mock.patch)
`mocker.spy(obj, "method")`	Spy on an existing method (track calls without replacing it)
`mocker.stub()`	Create a simple stub object
`mocker.assert_called_once()`	Assert a mock was called once (other assert_* helpers available)

Good to know

- pytest-mock is just a thin wrapper around unittest.mock.
- It doesn't replace any functionality — it makes it easier to use with pytest.
- Mocks are automatically cleared between tests.

Test smarter with pytest-mock

Mock what matters. Keep your tests clean.

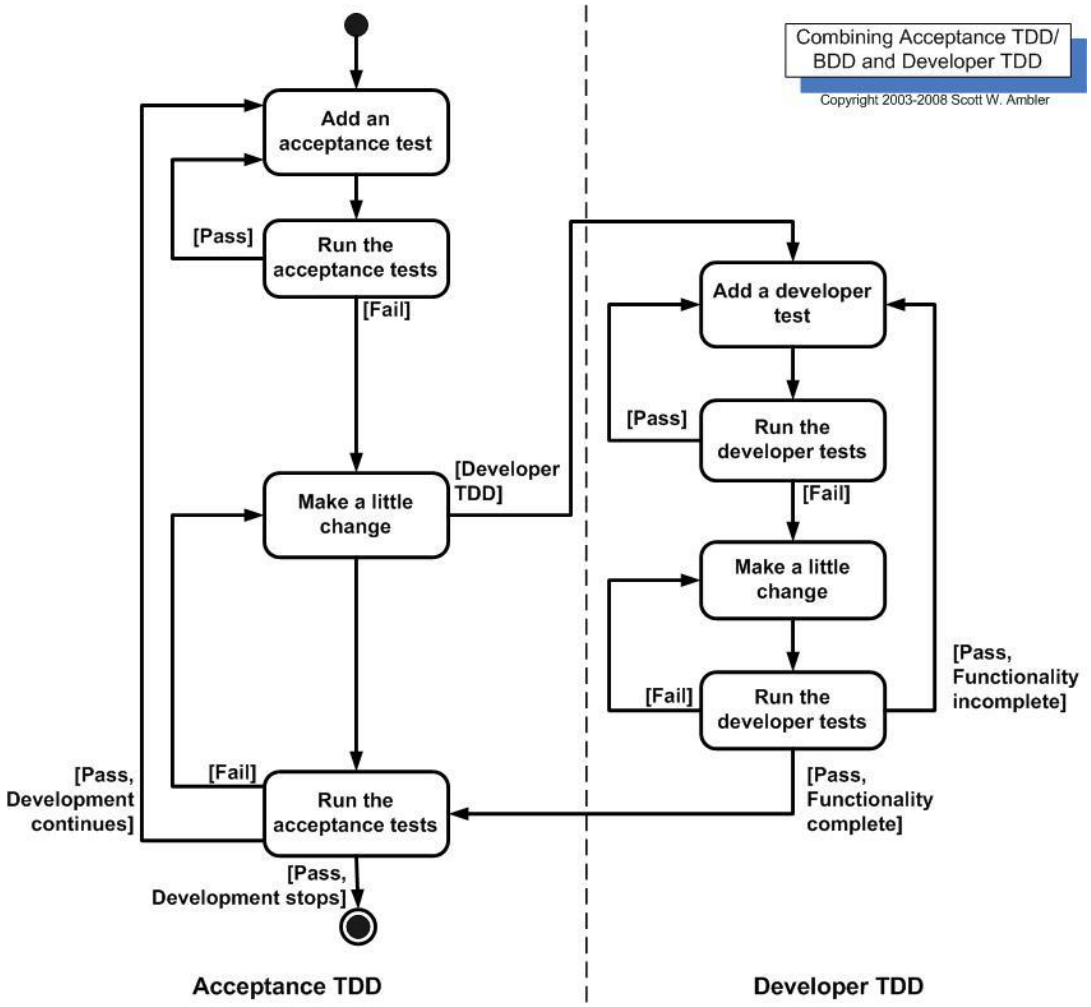


More reliable code, happier developers!

Microsoft Copilot

TDD

- Software design approach: Write tests before you write the actual code.
- Benefits:
  - Ensures your code is well-tested
  - Encourages modular, loosely-coupled code
  - Helps catch bugs early
  - Makes refactoring safer



Behavior-Driven Development (BDD): TDD extension, focus on the behavior of the system from the user's perspective, not just the code. Given-When-Then format.